

Lab Exercise 7 – Thread-Safe Counters Using **AtomicInteger** and **LongAdder**

Objective

The objective of this lab is to understand how Java provides thread-safe counters using **AtomicInteger** and **LongAdder**.

After completing this lab, students will be able to:

- Understand race conditions.
 - Use **AtomicInteger** for thread-safe counting.
 - Use **LongAdder** for high-performance counting.
 - Compare **AtomicInteger** and **LongAdder**.
 - Execute multiple threads safely without losing updates.
-

Scenario

A university maintains an online learning portal.

Every time a student logs in, the system increments a login counter.

Since hundreds of students may log in simultaneously, the application must count logins correctly without losing updates.

The university wants to implement two solutions:

- Using **AtomicInteger**
- Using **LongAdder**

Business Requirements

The system should:

- Simulate multiple students logging in simultaneously.
 - Increment a shared login counter.
 - Display the final login count.
 - Demonstrate thread-safe counting.
 - Compare AtomicInteger and LongAdder.
-

Project Structure

AtomicCounterLab

|

— src

|

— AtomicIntegerDemo.java

— LongAdderDemo.java

Only **two Java classes** are required.

Lab 1 – AtomicIntegerDemo.java

Objective

Learn how AtomicInteger safely updates a shared counter.

Complete Code

AtomicIntegerDemo.java

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicIntegerDemo {

    // Shared counter
    static AtomicInteger loginCounter = new AtomicInteger(0);

    public static void main(String[] args) throws InterruptedException {

        System.out.println("===== AtomicInteger Example =====");

        Runnable studentLogin = () -> {

            for (int i = 1; i <= 1000; i++) {

                loginCounter.incrementAndGet();

            }

            System.out.println(Thread.currentThread().getName() + " completed.");
        };

        Thread student1 = new Thread(studentLogin, "Student-1");
        Thread student2 = new Thread(studentLogin, "Student-2");
        Thread student3 = new Thread(studentLogin, "Student-3");
```

```
        student1.start();
        student2.start();
        student3.start();

        student1.join();
        student2.join();
        student3.join();

        System.out.println("\nTotal Student Logins : " + loginCounter.get());
    }
}
```

Expected Output

```
===== AtomicInteger Example =====

Student-1 completed.
Student-2 completed.
Student-3 completed.

Total Student Logins : 3000
```

(The order of thread completion may vary.)

Explanation of Main Code Blocks

1. Creating AtomicInteger

```
static AtomicInteger loginCounter = new AtomicInteger(0);
```

Explanation

Creates a thread-safe counter.

Initial value

0

2. Increment Counter

```
loginCounter.incrementAndGet();
```

Explanation

This method

- increases the counter by 1
- performs the operation atomically
- prevents race conditions

Example

0

↓

1

↓

2

↓

3

3. Starting Threads

```
student1.start();  
student2.start();  
student3.start();
```

Three students begin logging in simultaneously.

Each thread performs

1000 increments

4. Waiting for Completion

```
student1.join();  
student2.join();  
student3.join();
```

The main thread waits until all student threads finish.

5. Reading Counter

```
loginCounter.get();
```

Returns

3000

because

1000

+

1000

+

1000

=

3000

No updates are lost.

Lab 2 – LongAdderDemo.java

Objective

Learn how LongAdder improves performance when many threads update the same counter.

Complete Code

LongAdderDemo.java

```
import java.util.concurrent.atomic.LongAdder;

public class LongAdderDemo {

    static LongAdder loginCounter = new LongAdder();

    public static void main(String[] args) throws InterruptedException {

        System.out.println("===== LongAdder Example =====");

        Runnable studentLogin = () -> {

            for (int i = 1; i <= 1000; i++) {

                loginCounter.increment();

            }

            System.out.println(Thread.currentThread().getName() + " completed.");

        };

        Thread student1 = new Thread(studentLogin, "Student-1");
        Thread student2 = new Thread(studentLogin, "Student-2");
        Thread student3 = new Thread(studentLogin, "Student-3");

        student1.start();
        student2.start();
        student3.start();

        student1.join();
```

```
        student2.join();
        student3.join();

        System.out.println("\nTotal Student Logins : " + loginCounter.sum());
    }
}
```

Expected Output

```
===== LongAdder Example =====

Student-2 completed.
Student-1 completed.
Student-3 completed.

Total Student Logins : 3000
```

Explanation of Main Code Blocks

1. Creating LongAdder

```
static LongAdder loginCounter = new LongAdder();
```

Creates a high-performance thread-safe counter.

Unlike AtomicInteger, it maintains multiple internal counters.

2. Increment

```
loginCounter.increment();
```

Adds one to the counter.

Internally Java distributes updates across multiple counters when many threads are active.

3. Reading Total

```
loginCounter.sum();
```

Returns the combined value of all internal counters.

Example

Counter-1 = 700

Counter-2 = 800

Counter-3 = 1500

↓

Total = 3000